

How to use quirq infra

A Chromium-style CI/CD system for every quirq repo: the parts, how they fit together, how you work with it, and how it is named and governed.

WHAT QQ IS

One config repo decides; small tools apply it

Modelled on Chromium. infra-config declares every repo, builder, gate rule and channel as TOML. The tools that apply policy read it at a pinned commit.

Repo-agnostic. Core tools never name a language or product; a product repo writes one manifest and gets the rest generated.

Agents first. Agents may land clean reverts, dependency rolls and docs alone. Policy changes and stable promotion need suraj.

Public, GitHub-hosted. 13 infra repos, all Apache-2.0, serving two product repos: xo-space and innernet.

STATUS · 2026-10-05

Built and audited. v0 code merged in all 13 repos; repo rulesets applied (gate 6610664); suraj named owner of the org and every infra repo.

Live: generated CI on both repos, daily qq sync, lkgr at both mains, perf records, tree status open.

Not yet proven: a red PR refused, 7 unattended canaries, an automatic revert.

PHASE v0

v0: A base version of every repo, end to end on GitHub

Built; settings applied; exit test not yet passed

GOAL

All 13 infra repos exist, are public, sit behind their own gate and do a thin first version of their job. A product change is built from its manifest, gated on the exact merge result, watched after it lands and shipped to canary by agents daily.

SCOPE

Python 3.14 and Next.js 16 kinds; xo-space and innernet; GitHub merge queue + Actions behind an executor interface; only the canary channel live.

EXIT TEST · AS OF 2026-10-05

- 1 13 public repos, each with CODEOWNERS, a generated presubmit and a merge queue
partly: all have CODEOWNERS and rulesets; infra presubmits hand-written
- 2 Both product repos gated from their manifests; a red PR is refused
generated builders required; no red-PR refusal recorded yet
- 3 7 daily canaries with no human touch; bad canary held; rollback drill under 10 min
first scheduled run 2026-10-05 06:17 UTC; no canary record yet
- 4 A planted breaking commit is auto-reverted within caps, with a failure record
waits on the gardener App
- 5 Scorecard v0 computed by a script
test-pipelines scorecard workflow runs

PHASE v1

v1: Unattended canary, dev channel, fuzzing, failure tracking

Planned; starts after v0 validation

GOAL

The daily canary runs with nobody watching: it soaks, rolls itself back, fuzzes, files failure records and drafts postmortems. Dev opens with human approval. Flakes are handled, builds are hermetic and cached, and containers and docs repos join.

SCOPE

Headline items: canary soak + auto rollback, dev channel, Python/Node/API fuzzing, flake quarantine, shared REAPI cache, PostHog as a health signal, docs and wiki onboarded, one-command idempotent setup.

EXIT TEST

- 1 14 green daily canaries in a row, zero human touches
- 2 A planted bad canary is rolled back in its soak window, with failure record and cited postmortem draft
- 3 A planted fuzz crash becomes one deduplicated issue that closes only when replay passes
- 4 Dev promoted 4 times, each by a human owner approving agent evidence
- 5 A failure class seen twice in 14 days yields a new test, fuzz target or gate
- 6 Flake rate weekly; cache hit rate at least 90%; main red under 60 min a week for 4 weeks

PHASE v2

v2: Stable, speed, depth and Launchpad

Planned; stable and installers are low priority (P4)

GOAL

Everything that runs on GitHub can also run on Launchpad, quirk's own cloud. Remote execution and perf detection reach Chromium's shape, fuzzing gains depth, and agents hold the gardener, roller and release rotations.

SCOPE

Headline items: Launchpad backend and a gate without GitHub's merge queue, remote execution and autoscaled workers, cross-repo gating and rolls, perf bisection, stable promotion and staged rollout.

EXIT TEST

- 1 The same build runs on github and launchpad with matching digests, chosen only by config
- 2 Gate p50 under 10 min and p90 under 20 min for 4 weeks
- 3 A planted 10% perf regression is alerted and bisected to its culprit
- 4 8 weeks under suraj's intervention target, revert rate no worse than v1, policy changes owner-approved
- 5 P4: one stable promotion, a rollback drill under 10 min, xo-space installs follow a channel

COMPONENTS

13 infra repos, each one job

Repo	What it does	Chromium part
infra-config	All policy as TOML; qqcfg validates, generates, delivers	infra/config + lucicfg
depot	The qq CLI: sync, build, test, upload, try, land, status	depot_tools
sync	Manifest schema quirq-repo/1 and its only parser, qqsync	gclient / DEPS
recipes	One adapter per target kind (python-service, pytest, node-app)	recipes
toolchains	Builds CPython 3.14, Node 24 + pnpm; publishes to ghcr by digest	CIPD / 3pp
remote-build	Executor interface (local, github) and an action cache	RBE / REAPI
test-pipelines	JUnit sink, write-once results, verdicts, failure records	ResultDB, LUCI Analysis
gate	Required checks from config; rulesets and merge queue as code	LUCI CV
gardener	Tree status, failure grouping, bisection, capped auto-revert	Sheriff-o-Matic, Bisection
rollers	Dependabot config + land check; toolchain pin roller	AutoRoll
release	lkgr, channel pointers, rollback, daily canary	lkgr finder, promote.py
installer	Clients follow a channel instead of main	Omaha
perf	Build size and benchmarks per commit, a record never a gate	perf dashboard

infra-config · All policy as code

WHAT IT DOES

- Declares every repo, builder, gate rule, channel and cap as TOML under config/
- qqcfg validate checks schemas, references and invariants config cannot loosen
- qqcfg generate writes the builders; qqcfg deliver writes them into a product checkout
- Every tool reads it at a pinned commit, so a change reaches a tool only after a pin bump

YOU MEET IT AS

- qqcfg validate [--todos]
- qqcfg generate
- qqcfg deliver <repo> <checkout>
- config/repos.toml, pipelines.toml, gate.toml

v0 · AS OF 2026-10-05

- **Merged:** CFG-01..03, ORG-01; CFG-05 partly
- **main:** 310e326
- **Live:** Generated builders on both repos, sink 28f49a0 (xo-space #220, innernet #44 merged); suraj owns org and 13 repos
- **Open:** suraj's values (CFG-04), other area owners (ORG-02), compute ceiling (ORG-04)

NEXT IN v1

- Dev channel on
- Health thresholds
- Fuzz schedules and filing caps
- Onboard docs, wiki, containers

depot · The qq CLI

WHAT IT DOES

- One command for developers and agents, pinned per repo in infra/repo.toml
- Fetches toolchains, builds and tests through recipes, uploads and lands changes
- qq try returns a run ID at once and delivers the verdict later, so agents never block
- A daily e2e-sync job proves a fresh runner can sync both product repos

YOU MEET IT AS

- qq sync · qq fetch <url>
- qq build · qq test
- qq upload · qq try
- qq land · qq status (0 pass, 1 refused, 3 pending)

v0 · AS OF 2026-10-05

- **Merged:** DEP-01..04 and all audit fixes
- **main:** 741967c
- **Live:** e2e-sync daily at 06:17 UTC for xo-space and innernet
- **Open:** No depot v* tag can be cut until the release executor exists

NEXT IN v1

- qq roll
- qq channel (hold, release, roll back)
- qq failure and qq postmortem

sync · The manifest and its only parser

WHAT IT DOES

- Owns the manifest schema quirk-repo/1: qq version, toolchain pins by digest, typed targets
- qqsync is the one parser every other repo imports
- Pin check: every pin is a digest, and a fetch that does not match it fails
- A guard fails any other code that tries to parse the manifest itself

YOU MEET IT AS

- `infra/repo.toml`
- `qqsync validate`
- `qqsync show`
- `qqsync guard`

v0 · AS OF 2026-10-05

- **Merged:** SYN-01..04
- **main:** `d97e2f7`
- **Live:** Used by depot, recipes, gate, rollers, release and perf
- **Open:** none for v0

NEXT IN v1

- Mirroring: builds pass with upstream registries blocked
- Schema quirk-repo/1.1: fuzz targets, probes, deploy targets

recipes · One adapter per target kind

WHAT IT DOES

- One adapter contract per kind: fetch, build, test, run, deploy, bench (+ package, fuzz)
- Adapters: python-service, pytest, node-app (Next.js); the loader finds them, no registry
- Property tests run inside test
- Deploys to a canary test environment for release's canary

v0 · AS OF 2026-10-05

- **Merged:** REC-01..05 plus bench
- **main:** [db6ce0a](#)
- **Live:** Used by qq, release's canary and perf
- **Open:** Generated CI builders do not call it yet (interim commands)

YOU MEET IT AS

- `qq build / qq test` call it
- `release canary` and `perf` call it
- adding a language = one adapter module

NEXT IN v1

- container-image and static-docs adapters
- Python, Node and API fuzzing
- Real canary deploy targets

toolchains · Pinned, promoted toolchains

WHAT IT DOES

- Builds CPython 3.14.8 and Node 24.21.0 with pnpm 11.28.2
- Publishes each build to public ghcr, addressed by digest
- A build is staged, then promoted by PR into promoted.toml
- A promotion-gate workflow, pinned as an org-required check, guards promotions

YOU MEET IT AS

- promoted.toml
- toolchains.toml
- ghcr.io/quirq-ai/...@sha256

v0 · AS OF 2026-10-05

- **Merged:** TCH-01..03
- **main:** 44f8f95
- **Live:** suraj is code owner of the gate paths (#14); promotion-gate runs on PRs and the queue (#15)
- **Open:** promotion-gate becomes required at the next settings apply (gate #25)

NEXT IN v1

- Reproducibility check
- Container toolchain

remote-build · Where actions run, and their cache

WHAT IT DOES

- An executor interface with two backends: local and github
- A local action cache keyed by action digest, with fallback counters
- Launchpad plugs into the same seam later

v0 · AS OF 2026-10-05

- **Merged:** RBE-01, RBE-02
- **main:** [62b2f04](#)
- **Live:** Interface and cache ready
- **Open:** qq does not call it yet

YOU MEET IT AS

- `qqrbe exec/selftest --backend local|github`
- `recipes action shape`

NEXT IN v1

- Shared REAPI cache (v2: remote execution, autoscaled workers)

test-pipelines · Results and verdicts

WHAT IT DOES

- A sink in every builder stores each JUnit result in a write-once store
- Verdict: retry, then compare with the base commit
- Failure records with an issue mirror; gardener writes one per revert
- A scorecard workflow collects results, perf bundles and gate timing

YOU MEET IT AS

- `sink` step in each `qq-*.yaml`
- `qqresults`
- `scorecard workflow`

v0 · AS OF 2026-10-05

- **Merged:** TST-01..04 and audit fixes
- **main:** [a642399](#)
- **Live:** Product results stored via sink 28f49a0, the v0 pin
- **Open:** Retry-then-compare is off in product builders

NEXT IN v1

- Flake detection and quarantine with expiry
- Exoneration of known flakes
- New-test flakiness check
- Failure classes and recurrence

gate · What must pass before a change lands

WHAT IT DOES

- Computes each repo's required checks from infra-config and its manifest
- Rulesets and the merge queue as code, applied by one command
- Agnosticism guard: no core file may name a language or tool
- Gate timing (queue to verdict) into the scorecard

YOU MEET IT AS

- qgate (0 pass, 1 refused, 2 undecided, 3 not onboarded)
- settings/github.toml
- scripts/apply.sh one-liner

v0 · AS OF 2026-10-05

- **Merged:** GAT-01, 02, 04 and ORG-03
- **main:** c372136
- **Live:** Repo rulesets applied at 6610664; org rulesets off (GitHub Free)
- **Open:** GAT-03 waits on owners; re-apply for promotion-gate

NEXT IN v1

- Result reuse
- Admission bar enforcement
- Tree status in the gate

gardener · Keeps main green

WHAT IT DOES

- Checks the tree every 5 minutes and publishes status per repo
- Groups failures, finds the culprit by bisection
- Opens clean revert PRs within the cap of 10 per 24 h
- Writes a failure record and postmortem stub per revert

YOU MEET IT AS

- tree-status branch: `status/<repo>.json`
- ledger branch (after a manual bootstrap run)
- `auto_revert.toml` caps

v0 · AS OF 2026-10-05

- **Merged:** GAR-01..04 and audit fixes
- **main:** `bf7d24d`
- **Live:** Tree status published; both repos open
- **Open:** Its GitHub App, not yet created: until then it reports reverts but opens none

NEXT IN v1

- Test-failure reverts
- Revert precision
- Postmortem drafting agent
- Canary bisection

rollers · Keeps dependencies and toolchains current

WHAT IT DOES

- Ships Dependabot config to each product repo
- qq-roll-land checks that a roll is a clean patch or minor bump
- Rolls toolchain pins from promoted.toml as PRs from the quirker App
- Weekly toolchain rotation

YOU MEET IT AS

- `.github/dependabot.yml`
- `qq-roll-land.yml`
- `branch qq-roll/toolchains`

v0 · AS OF 2026-10-05

- **Merged:** ROL-01, ROL-02 and lockfile tree check
- **main:** 4285aa4
- **Live:** Dependabot config on both repos; first roll innernet #43 (TS 5 → 7) merged
- **Open:** Auto-land is off: a person merges every roll

NEXT IN v1

- Weekly rolls for every pin (v2: cross-repo rolls with canary feedback)

release · lkgr, channels and the daily canary

WHAT IT DOES

- Moves lkgr to the newest all-green main commit every 10 minutes
- Channel pointers with digests, and rollback
- Daily canary at 06:17 UTC: build, test, deploy and probe, then move channels/canary
- Writes a daily canary report; a watchdog starts a canary GitHub dropped

YOU MEET IT AS

- release-state branch
- channels.json (qq-channels/1, after the first canary)
- qq channel rollback

v0 · AS OF 2026-10-05

- **Merged:** REL-01..04 and audit fixes
- **main:** 83ef917
- **Live:** lkgr at both repos' current main in release-state
- **Open:** No git refs until the release executor App exists (after v0)

NEXT IN v1

- Canary soak and automatic rollback
- Canary fuzz stage
- Dev channel
- Escalation

installer · Installs follow a channel, not main

WHAT IT DOES

- Reads release's channels.json for a repo and channel
- Resolves the commit and digest, then checks out and verifies exactly that
- v0 scope: xo-space test installs only, never real users' machines

YOU MEET IT AS

- `qqinstall resolve --repo xo-space --channel canary`
- `qqinstall checkout · verify`
- `exit 0` ok, `1` mismatch, `2` error, `3` not published

v0 · AS OF 2026-10-05

- **Merged:** INS-01, INS-02 with audit fixes
- **main:** `d39f302`
- **Live:** Resolves nothing yet (exit 3)
- **Open:** Waits on first canary and test machines; live-manifest check likely red until then

NEXT IN v1

- Test installs follow dev (v2: xo-space follows a channel)

perf · Performance on every commit

WHAT IT DOES

- Records a benchmark per main commit for both repos, and innernet's build size
- Stores raw numbers with unit and runner type as test-pipelines bundles
- A record only in v0: never a gate or an alert

YOU MEET IT AS

- perf-data branch
- infra-config config/perf.toml
- benchmarks: xo-space-server-start, innernet-search

v0 · AS OF 2026-10-05

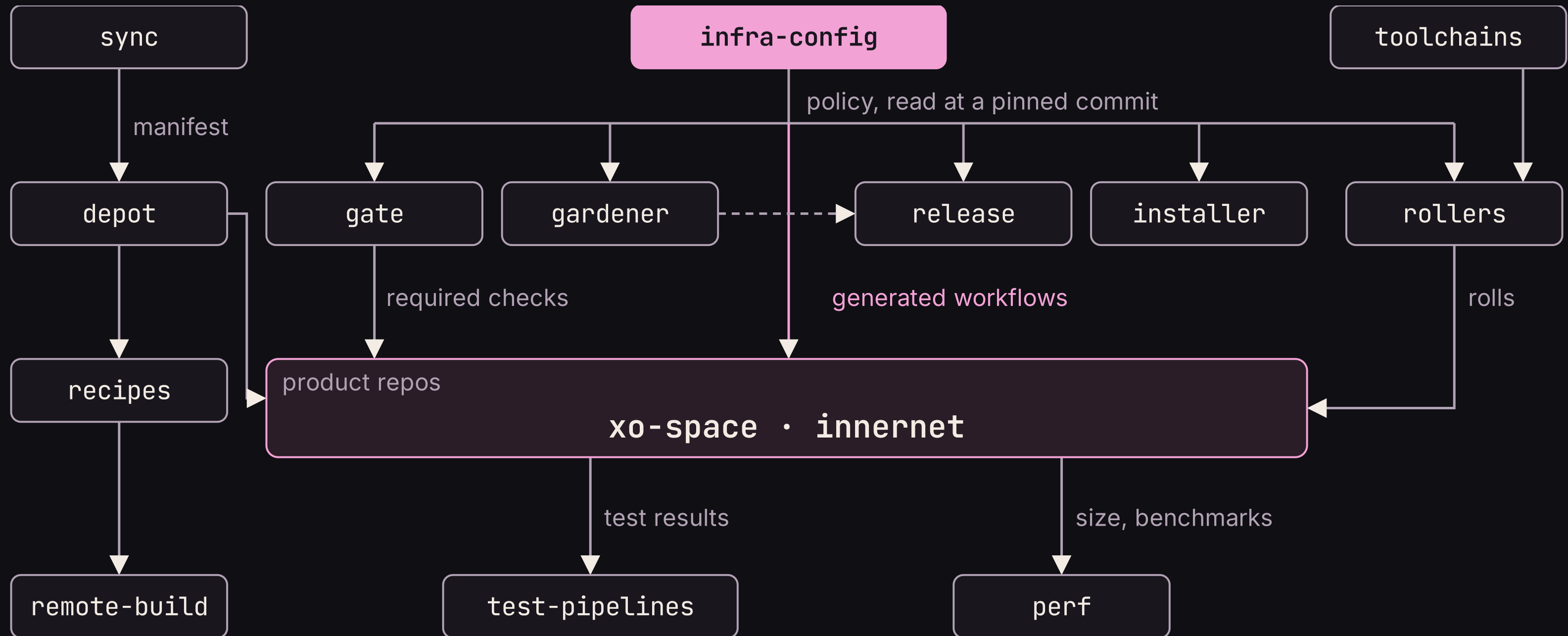
- **Merged:** PRF-01, PRF-02 and audit fixes
- **main:** d1d9765
- **Live:** Recording main commits on perf-data (newest record 2026-10-05 02:27 UTC)
- **Open:** One measurement per commit on shared runners, no baseline

NEXT IN v1

- Trend and threshold alerts
- Bundle-size budget

HOW THEY INTERACT

Policy flows down, results flow back



Dashed: gardener tells release which main commits are green, and release moves lkgr. Full list on the next slide.

HOW THEY INTERACT

Every link is a pinned commit, never copied code

From	To	What flows
infra-config	gate, gardener, release, installer, rollers	Policy via qqcfg at each tool's pin (pins.toml; rollers: infra-config.commit)
infra-config	product repos	Generated qq-*.yml workflows, written by qqcfg deliver into a PR
sync	depot, recipes, gate, rollers, perf	The manifest, read only through qqsync
toolchains	rollers, then product repos	promoted.toml digests, turned into pin-roll PRs
depot	recipes; gate	qq build and test run through recipes; qq land and status run the pinned qqgate
recipes	release, perf; remote-build	Canary and perf builds run through recipes; remote-build runs its actions (not yet called)
gate	product repos	Required checks and rulesets, once suraj applies them
product CI, perf	test-pipelines	JUnit through the sink; perf result bundles; the scorecard
test-pipelines	gardener	Failing tests per run; gardener writes one failure record back per revert
gardener	release	What counts as a green post-submit (qqgarden), used to move lkgr
release	installer, depot	channels.json for installs; the qq channel rollback subcommand

From a fresh clone to an open PR

1 Set up once

Clone depot and put its bin on PATH. In a product repo, qq runs the version pinned in infra/repo.toml (both pin 0.1.0, depot main).

2 Get toolchains: qq sync

Fetches every pinned toolchain from public ghcr, checks its digest, links it under .qq/toolchains/. Runs daily for both repos.

3 Build and test: qq build, qq test

Recipes run the targets and leave JUnit and results.json in .qq/out.

4 Open a PR: qq upload or a plain push

The <repo>-presubmit check runs a drift check, then fetch, build and test, then stores every result in test-pipelines.

From a verdict to a channel

qq try returns a run ID at once and delivers the verdict later to \$QQ_HOME/verdicts/, so an agent never holds a call open.

qq status gives it on demand: exit 0 pass, 1 refused, 3 pending.

qq land queues the PR once the gate passes; the merge queue re-tests the exact merge result and squash-merges.

After it lands, the post-submit builder runs on every main commit and gardener checks the tree every 5 minutes, proposing a revert if main goes red.

Dependencies move by Dependabot PRs, checked by qq-roll-land, and toolchain roll PRs from quirker. A person merges every roll in v0.

Ship. release moves lkgr to the newest all-green main commit; the daily canary at 06:17 UTC builds, tests, deploys and probes it before moving channels/canary.

qqinstall resolve --repo xo-space --channel canary gives the commit and digest to install; exit 3 until the first canary ships.

Measure. perf records build size and a benchmark per main commit.

Today: gate's repo rulesets (merge queue, required checks) were applied at gate 6610664. Reverts and git refs for lkgr and channels wait on bots that come after v0; the first canary was scheduled for 2026-10-05 06:17 UTC.

Write one manifest; the rest is generated

What	Lands as	Live today
Manifest: qq version, toolchain pins by digest, targets	infra/repo.toml	both repos
Presubmit builder, the required check	qq-<repo>-presubmit.yml	both repos
Post-submit builder on every main push	qq-<repo>-postsubmit.yml	both repos
Drift check against infra-config	a step in each qq-*.yml; org qq-drift	step yes; org rule off (Free plan)
Merge queue, required checks, squash only	the qq-main ruleset (gate)	applied (gate 6610664)
Every test result stored	a sink step in each builder	yes
Tree status, culprits, revert PRs	gardener's tree-status branch	status yes; reverts later
Dependabot + land check; toolchain rolls	dependabot.yml, qq-roll-land.yml	yes, auto-land off
Build size and benchmark per commit	perf's perf-data branch	yes
lkgr, daily canary, channels/canary	release-state record; refs later	lkgr yes; canary not yet recorded
Test installs follow canary	qqinstall checkout and verify	after first canary

POLICIES

Every policy is a TOML file in infra-config/config

File	Decides · value today
gate.toml	Who may land what: reverts, rolls and docs need a green gate; code needs an owner; policy needs suraj
gate.toml	Squash merges; a blocking builder needs p50 under 15 min and must finish under 40
pipelines.toml	Builders per repo: blocking presubmit, post-submit mirror, canary deploy (run by release)
kinds.toml	Toolchains and target kinds: Python 3.14, Node 24 with pnpm
flakes.toml	Retries and flakes, read by test-pipelines' verdict
auto_revert.toml	At most 10 reverts per rolling 24 h; auto_land_repos empty, so every revert is proposed
rollers.toml	Dependabot for pip and npm; weekly toolchain rotation
channels.toml, health.toml	canary: agents, daily; dev: a human owner after 24 h soak and 3 green canaries; stable: suraj
repos.toml	Every repo: source, kinds, manifest path, channels, owners, deploy target
gate settings/github.toml	Rulesets as code: qq-main (merge queue, checks), ref and tag locks; repo rulesets applied at 6610664; org rulesets and Dependabot rules off

Rules config cannot loosen live in tools/qqcfg.py: stable needs suraj, canary is agents-only, at most 10 auto-reverts a day, no secrets for builders that run PR code, public repos only.

POLICIES

How a policy change reaches a repo

1 · EDIT

Change config/<area>.toml in a PR. CI runs qqcfg validate: schemas, references, invariants, generated/ up to date.

2 · APPROVE

CODEOWNERS names suraj on every policy path. Squash-merge.

3 · DELIVER

qqcfg deliver <repo> <checkout> writes the workflows into a product-repo checkout, opened as a PR; drift checks fail hand edits.

4 · PIN BUMP

Each tool sees the change only after its own infra-config pin is bumped.

Repo settings are code too: edit gate's settings/github.toml, merge, then an org owner runs one command

```
( mkdir -p ~/qq-apply && cd ~/qq-apply && rm -rf qq-gate && git clone -q https://github.com/quirq-ai/gate qq-gate && cd ./qq-gate && git checkout -q <COMMIT> && scripts/apply.sh )
```

It shows a dry run, writes only what you approve at its prompts, and is safe to re-run. Use the gate commit you are given; details in gate's docs/apply-settings.md.

The name says the area, the description says the job

Thing	Convention	Examples
Product repo	its brand name; a part is <product>-<part>	xo-space, innernet
One-repo area	the area word	website, marketing, docs
Multi-repo area	<area>-<thing>, the thing being its job	infra-<thing> (new infra repos)
The 13 infra repos	keep their names for v0; infra-* rename is optional, later	gate, release
Spelling	lowercase, words joined by -, no versions or years	
Description	<area>: <job in one line>	website: code and deploy of...
Area and owner	org properties area, owner-bot, owner, stage; select by property, never by name	area = infra
Bots	one GitHub App per tool or area: quirq-<repo or area>	quirq-gardener; rollers: quirqer
qq's own files	the qq prefix	qq CLI, .qq/, qq-*.yml
Builders and checks	<repo>-<pipeline>; the job name is the required check	xo-space-presubmit
Work items	V<version>-<AREA>-<nn>	V0-GAT-01, V1-PH-01
Release refs	lkgr and channels/<channel>	channels/canary

Who decides, who acts

People

suraj is the policy owner: policy paths, stable promotion and repo settings.

org.toml names suraj owner of the org and all 13 infra repos (infra-config 310e326). Other config areas and the product repos in repos.toml still have empty owners lists (V0-ORG-02).

Every infra repo's CODEOWNERS names suraj on its policy and trust paths, with no catch-all.

```
qqcfg validate [--todos]
```

lists every empty owner list and rotation; --todos adds all open TODOs, 3 of them for v0.

Agents and bots

Agents may land alone only clean reverts, dependency rolls and docs, and promote to canary.

One GitHub App per tool. Live: quirqer (rollers). After v0, designed with suraj: the gardener App and the release executor.

A bot works only in repos whose owner-bot property names it.

GitHub does not let a PR's author approve it: suraj can approve agent PRs (opened as quirq-gates), but suraj's own PRs need a second owner or a bot.

ADDING A NEW REPO

Two PRs and a settings run; no tool code changes

The repo must be public in quirk-ai, named by the conventions, and use a known kind: python-service, pytest or node-app.

Where	Change	Approves
infra-config repos.toml	one [[repo]] block: source, kinds, manifest, channels, owners	suraj
infra-config pipelines.toml	presubmit and post-submit mirror; a canary builder if it ships canary (validate requires them)	suraj
the new repo	infra/repo.toml, workflows from qqcfg deliver, Dependabot files	repo owner
gate settings/github.toml	a [[repo]] entry, then the apply one-liner (org rulesets are off on GitHub Free)	suraj
GitHub	add to the gardener and rollers App installs; Dependabot and auto-merge on	suraj (admin)

Follows by itself once tools bump their pin: required checks, tree status, channels.

Still a hand edit: the test-pipelines scorecard and perf matrix list repos by name.

New language: a kind in kinds.toml, one recipes adapter, one toolchains build.

The biggest gap is live proof

Limit	Effect	Clears when
Enforcement not proven live	Repo rulesets applied, but no red-PR refusal recorded; org rulesets off on GitHub Free	Red-PR validation in both repos
promotion-gate not required	toolchains' promotion check stays advisory	Settings re-apply from gate c372136
No gardener App or release executor	No reverts opened; no lkgr or channel git refs; no depot v* tags	Per-tool bots after v0
Owners partly named	Other config areas and product repos have none; GAT-03 waits	suraj names owners or a bot
CI uses interim steps	Builders use setup-python/node, not recipes; retry-then-compare off	v1
Nothing lands alone	auto_land_repos empty; roll auto-land off	Policy and gate settings PRs
Canary not proven live	First scheduled run 2026-10-05 06:17 UTC; no record yet	Canary validation, test machines
Thin signals	perf: one measurement per commit on shared runners; no PostHog yet	v1
Narrow platform	GitHub only; toolchains for linux-x86_64 only	Launchpad (v2); more platforms
Open values	Canary hour, how the revert cap counts, monthly compute ceiling	suraj sets them

WHAT'S NEXT

Finish v0, then build the bots with suraj

TO CLOSE v0

1. suraj's values (CFG-04) and the remaining owners (ORG-02), which unblock GAT-03
2. Re-apply gate settings so toolchains requires promotion-gate
3. Validate: a red PR is refused
4. Validate: the daily canary, including its no-op path
5. Validate: gardener tree status stays right

AFTER v0

Per-tool GitHub Apps: gardener, release executor

Builders switch to recipes and promoted toolchains

Agents land rolls and reverts alone, within caps

One-command, idempotent setup (V1-ORG-01)

New repos website and marketing, named by the org conventions

Optional: rename infra repos to infra-*

Done since 2026-10-04: the settings run, owners for the org and infra repos, and the 9123fa9 redelivery (xo-space #220, innernet #44).

SOURCES

Where every status on these slides comes from

REPOS AT main, READ 2026-10-05
github.com/quirq-ai/<repo>/tree/<commit>

depot	741967c	sync	d97e2f7
recipes	db6ce0a	toolchains	44f8f95
remote-build	62b2f04	infra-config	310e326
gate	c372136	test-pipelines	a642399
gardener	bf7d24d	rollers	4285aa4
perf	d1d9765	release	83ef917
installer	d39f302	xo-space	14b21a4
innernet	8f383a3		

State branches, fetched 2026-10-05: release `release-state` at f89a944 (lkgr pointers, no canary record); gardener `tree-status` at 6733e4c; perf `perf-data` at 28628f3.

Settings as the repos record them (gate README and settings/github.toml); live GitHub settings were not read.

Phases v1 and v2 are plans: quirq infra plans vision/quirq-infra/v0.md, v1.md and v2.md (project files, 2026-10-03; v1 updated 2026-10-04).

Naming: vision/org/naming.md (project files, adopted 2026-10-04).

Write-ups: quirq-ai/research, infra/output/report: the phases, v0 status, setup and build-and-run reports.